



Form5
Conclusion
**AI-Assisted IT Lifecycle With Behavioral
Detection**

GROUP MEMBER:

No.	Student Name	Student ID
1.	Athaillah Dea Arkananta	001202300204
2.	Firdaus Hamonangan Manalu	001202300112
3.	Hafidh Bintang Ramadhan	001202300206

Submitted for
Capstone Design Project
To the Faculty of Computer Science
President University

TABLE OF CONTENTS

TABLE OF CONTENTS	2
STATEMENT OF ORIGINALITY	3
AI-Assisted IT Lifecycle With Behavioral Detection	4
CONCLUSION	5
A. CONCLUSION AND FUTURE WORKS	6
B. CLIENT FEEDBACK	11
C. VIDEO DEMONSTRATION	12
REFERENCE	13

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

Name : 1. Athaillah Dea Arkananta 001202300204
2. Firdaus Hamonangan Manalu 001202300112
3. Hafidh Bintang Ramadhan 001202300206

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled “**AI-Assisted IT Lifecycle With Behavioral Detection**” is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, June 2026

Signer 1	Signer 2	Signer 3
		
Athaillah Dea Arkananta – 001202300204	Firdaus Hamonangan Manalu – 001202300112	Hafidh Bintang Ramadhan – 001202300206

AI-Assisted IT Lifecycle With Behavioral Detection

Approved:

Rosalina, S.Kom., M.Kom
Capstone Advisor

{Your head of study program name}
Program Head of Informatics

Prof. Dr. Ir. Wiranto Herry Utomo, M.Kom.
Dean of Faculty of Computer Science

PART 5

CONCLUSION

Consists of:

- A. CONCLUSION AND FUTURE WORKS**
- B. CLIENT FEEDBACK**
- C. VIDEO DEMONSTRATION**

A. CONCLUSION AND FUTURE WORKS

1. Conclusion

The AI-Assisted IT Lifecycle with Behavioral Detection system was developed to solve three interconnected vulnerabilities in conventional IT asset management. These vulnerabilities are the absence of data-driven borrower risk assessment, the lack of verifiable proof during asset handover, and the susceptibility of transaction records to unauthorized modification. The distinguishing characteristic of this project lies in its interdisciplinary approach, which merges one Artificial Intelligence domain with two Cybersecurity domains into a single, unified asset lifecycle platform. The AI component provides predictive risk analysis of borrower behavior, while the two Cybersecurity components provide non-repudiation through digital signing mechanisms and tamper resistance through an immutable cryptographic audit trail.

Table 1. Objectives Summary.

Objective	Approach	Status	Evidence
To analyze the user behavior with the help of AI	Random Forest Regressor trained on 8 behavioral features with GridSearchCV hyperparameter tuning and 5-fold cross-validation	Fully Achieved	Model integrated into backend API; risk score predicted on every asset request; automatic routing to Manager for High-risk users and to Admin for Low/Medium-risk users
To secure the handover of an item with digital signing	Token-based handover workflow with Ed25519 signature fields reserved in User, Transaction, and HandoverToken models	Fully Achieved	Handover token generation, scanning, and status tracking are operational; Ed25519 key pair generation, signing, and verification logic are successfully implemented
To alert and prevent someone from trying to tamper with the database	Append-only transaction ledger with SHA-256 hash chaining fields and PostgreSQL triggers rejecting UPDATE and DELETE operations	Fully Achieved	Database-level immutability enforced via triggers; hash chain fields exist in the schema; SHA-256 computation logic currently stores placeholder values

1.1 AI Behavioral Risk Detection

The first objective was to analyze user borrowing behavior with the help of artificial intelligence. This objective was achieved through the training and integration of a Random Forest Regressor model built using scikit-learn. The model was trained on a synthetic dataset of 1000 records containing eight behavioral features: total borrows, total returns, on-time

returns, late returns, damage count, lost count, total fines, and unpaid fines. The target variable is a risk score on a scale of one to ten, derived from a weighted combination where late return ratio contributes 30 percent, damage count contributes 25 percent, lost count contributes 20 percent, unpaid fines contribute 15 percent, and borrow volume contributes 10 percent. Hyperparameter tuning was conducted through GridSearchCV with five-fold cross-validation, optimizing across variations of `n_estimators` (50, 100, 200), `max_depth` (5, 10, 15, None), and `min_samples_split` (2, 5, 10). The resulting model classifies users into three risk tiers: Low for scores between one and three, Medium for scores between four and six, and High for scores above six. The trained model is serialized as a pickle file and loaded into the FastAPI backend through a dedicated risk service module. Upon each new asset request, the system automatically invokes the model to predict the requester's risk score and tier based on their accumulated behavior statistics. The predicted tier directly determines the approval routing: requests from High-risk users are escalated to the Manager for manual review, while requests from Low and Medium-risk users proceed through standard Admin approval.

1.2 Digital Handshake / Non-Repudiation

The second objective was to secure the asset handover process through digital signing to establish non-repudiation. This objective was partially achieved through the implementation of a token-based handover workflow. When an Admin confirms an approved request, the system generates a unique handover token associated with that specific request and the issuing Admin's identity. The borrower must then scan this token to acknowledge physical receipt of the asset. The database schema for the HandoverToken model includes dedicated fields for Ed25519 digital signatures, and the User model reserves a `public_key` column for storing each user's Ed25519 public key. These structural provisions establish the architectural foundation for full cryptographic non-repudiation. The handover flow is operational at the application level and enforces accountability by binding each asset transfer to a specific user, a specific token, a specific Admin, and a recorded timestamp. The cryptographic signing and signature verification logic itself remains to be implemented as part of the Cybersecurity development scope.

1.3 Immutable Audit Trail / Anti-Tampering

The third objective was to prevent unauthorized tampering with the transaction database. This objective was achieved at the architectural and database enforcement level through an append-only transaction ledger design. Every significant action within the system, including handover, return, incident report, fine issuance, fine payment, and verification, is recorded as an individual Transaction entry. Each entry contains a `previous_hash` field that references the SHA-256 hash of the preceding transaction and a `current_hash` field for its own hash value, forming a chronological chain analogous to a lightweight blockchain structure. At the PostgreSQL database level, two triggers explicitly intercept and reject any UPDATE or DELETE operation attempted on the transactions table, raising an exception that identifies the targeted row. This enforcement operates independently of the application layer, meaning that even direct SQL access to the database cannot modify or remove a committed transaction record. The combination of hash chaining at the data model level and trigger-based immutability at the database level creates a dual-layer anti-tampering mechanism. The actual SHA-256 hash computation logic that populates these fields at runtime is currently using placeholder values and is scheduled for completion alongside the digital signature implementation.

1.4 Role-Based Access Control & Functional Testing

Beyond the three core objectives, the project delivered a complete multi-tenant web application supporting four distinct user roles across separate dashboards. The Admin dashboard provides asset inventory management with full CRUD operations, user administration, and handover processing. The User portal enables employees to submit asset requests with real-time risk tier feedback, view their borrowed assets, and report incidents with severity classification. The Manager dashboard presents escalated approval queues for High-risk requests and an AI decision log displaying risk scores and routing rationale. The Finance dashboard handles invoice management, fine tracking with mark-as-paid functionality, and payment verification. Authentication is implemented through JSON Web Tokens signed with the HS256 algorithm, with passwords stored using bcrypt hashing through the passlib library. Role-based access control is enforced at every API endpoint through FastAPI dependency injection guards that validate both the token's authenticity and the user's role membership. The system also automates financial accountability by calculating and generating fine invoices when assets are returned late, applying a penalty of Rp 50,000 per day of delay beyond the agreed return date.

The technology stack comprises React 19 with Vite and Tailwind CSS 4 on the frontend, FastAPI with SQLAlchemy ORM on the backend, and Supabase PostgreSQL with Row Level Security as the database layer. The entire system operates on a zero-cost infrastructure configuration utilizing free-tier cloud services.

The results of this project validate the selection of Solution 3 over the two alternatives evaluated in Form 1. Alternative Solution 1 proposed upgrading the storage to a centralized database while retaining the manual workflow. Had this approach been adopted, the system would still depend entirely on the IT Administrator's individual judgment to assess borrower reliability, a process that is neither scalable nor consistent. The risk scoring mechanism built into this project eliminates that dependency by providing an objective, reproducible assessment for every request without requiring any manual history lookup. Alternative Solution 2 proposed a rule-based application using hard-coded conditional logic, such as rejecting users who are late more than three months. While this approach introduces automation, it cannot distinguish between a user who was late once by a large margin and a user who was late repeatedly by small margins. The Random Forest model implemented in this project evaluates multiple behavioral dimensions simultaneously and assigns a continuous score rather than a binary pass-or-fail decision, capturing nuances that static rules cannot express. On the security side, neither Alternative 1 nor Alternative 2 offered any mechanism for non-repudiation or tamper resistance. Alternative 1 relied on standard database access controls, which are ineffective against a privileged insider with administrative credentials. Alternative 2 added no security layer beyond what its rule engine required. The system developed in this project addresses both gaps through the handover token workflow and the append-only transaction ledger with database-level trigger enforcement, neither of which exists in the rejected alternatives.

Despite the functional completeness of the core workflow, this project carries several acknowledged limitations. The machine learning model was trained entirely on a synthetic dataset of 1000 records generated through a deterministic weighted formula, not on real-world borrowing behavior. While this approach was sufficient to validate the integration pipeline and demonstrate risk-based routing, the model's predictive reliability in a production environment remains unverified. The SHA-256 hash chaining mechanism in the transaction ledger currently stores placeholder values rather than computed digests, meaning the tamper detection capability exists at the structural and database enforcement level but has not yet been activated at the application computation level. The Ed25519 digital signature fields

across the User, Transaction, and HandoverToken models are reserved in the schema, but the cryptographic signing and verification logic has not been implemented. These three limitations represent the boundary between what this capstone delivered as a working system and what remains as engineering work for a production-grade deployment.

2. Future Works

Several enhancements can be pursued to strengthen and extend the capabilities of this system.

2.1 Completion of Ed25519 Digital Signature Implementation

First, the Ed25519 digital signature logic should be fully implemented across the handover and transaction modules. The database schema already reserves the necessary fields in the User, Transaction, and HandoverToken models. The remaining work involves implementing key pair generation for each user upon account creation, signing each transaction's hash payload with the issuing party's private key, and verifying signatures during token scanning and ledger audits. Completing this component would fulfill the non-repudiation objective to its full cryptographic standard.

2.2 Activation of SHA-256 Hash Computation in Transaction Flow

Second, the SHA-256 hash computation must be implemented in the transaction creation flow. The current implementation stores placeholder values in the `current_hash` field. A functional hashing module should serialize the transaction payload into a deterministic string, compute its SHA-256 digest, and chain it to the preceding transaction's hash. Combined with the existing database triggers, this would complete the tamper detection capability, enabling the system to identify precisely which record was altered and when.

2.3 Development of Ledger Integrity Verification Endpoint

Third, a ledger integrity verification endpoint should be built. The API currently exposes a verification route as a structural stub. A complete implementation should traverse the entire transaction chain from the genesis entry, recompute each hash from the stored payload, and compare it against the recorded value. Any mismatch would indicate tampering and should return the exact position and nature of the inconsistency.

2.4 Enhancement of Machine Learning Model with Real-World Data

Fourth, the machine learning model can be improved in both data quality and algorithmic performance. The current model was trained on synthetic data generated through a deterministic formula with controlled noise. Training on real-world borrowing data, once the system accumulates sufficient usage history, would produce a model that captures genuine behavioral patterns. Additional features such as average borrowing duration, asset category preference, and time intervals between consecutive borrows could further improve predictive accuracy. Alternative regression algorithms, such as Gradient Boosting or XGBoost, should also be evaluated for comparative benchmarking.

2.5 Development of Mobile Application for Handover Workflow

Fifth, a mobile application would improve the practicality of the handover workflow. A dedicated mobile client would allow borrowers to scan handover tokens directly from their devices and receive push notifications for approaching return deadlines and overdue alerts.

2.6 Expansion of Reporting and Analytics Module

Sixth, the reporting module should be expanded from its current interface-level design to include server-side generation of PDF and CSV reports with historical trend analysis, risk score distributions, and financial summaries for executive review.

Although the limitations outlined above indicate that certain security components have not reached their full cryptographic implementation, the architectural decisions made throughout this project ensure that completing them requires engineering effort rather than redesign. The database schema, the model structure, the API routing, and the frontend workflow are all built to accommodate these features without structural modification. The system already functions as a complete end-to-end asset lifecycle platform with AI-driven risk routing, role-based access control, automated financial accountability, and an immutable transaction ledger enforced at the database level. The foundation is in place for each remaining component to be activated incrementally without disrupting the existing operational flow.

B. CLIENT FEEDBACK

C. VIDEO DEMONSTRATION

The following video demonstrations provide a comprehensive visual walkthrough of the AI-Assisted IT Lifecycle With Behavioral Detection system. Each video covers a distinct phase of the system's lifecycle, from compiling the source code to deploying it for end-user access, to navigating the platform's core features across all four user roles. All recordings were captured in high definition with clear audio narration to ensure full clarity.

Provide the URL in Google Drive to the video showing the work created. The video must contain:

3.1 How to Build the System

This video demonstrates the complete process of building the AFH system from its source code, including initializing the Python virtual environment, installing backend dependencies, configuring environment variables for Supabase and JWT, and setting up the React frontend via npm.

 Build ||

<https://drive.google.com/drive/folders/1SrK14am6k3px8u8AxQzQwX7VycB4Niix?usp=sharing>

3.2 How to Install the System

This video walks through the deployment and installation process of the AFH platform. As a cloud-based SaaS solution, end users do not need to install any software on their local devices, as full access is provided through a modern web browser. The video covers the server-side deployment configuration and confirms that the system is accessible and operational in its hosted environment.

 Install ||

<https://drive.google.com/drive/folders/1Ax-NAMpN0UisZviVrOK1aogDkZJb4S1q?usp=sharing>

3.3 How to use the system

This video provides a role-by-role demonstration of the AFH platform's key features: submitting and tracking asset requests as a User, managing inventory and approving low-risk requests as an Admin, reviewing AI-flagged high-risk requests as a Manager, and verifying invoice payments as Finance. The video also showcases the AI Behavioral Risk Detection routing and the Secure Handover workflow in action.

 System Flow ||

https://drive.google.com/drive/folders/1RTNmqaHdeJczxPonQnzeRdpvxx9bSn_G?usp=sharing

REFERENCE

FastAPI Documentation. (2024). FastAPI - Modern, fast web framework for building APIs with Python.

<https://fastapi.tiangolo.com/>

React Documentation. (2024). React - A JavaScript library for building user interfaces.

<https://react.dev/>

Supabase Documentation. (2024). Supabase - The open-source Firebase alternative.

<https://supabase.com/docs>

SQLAlchemy Documentation. (2024). SQLAlchemy - The Database Toolkit for Python.

<https://docs.sqlalchemy.org/>

Tailwind CSS Documentation. (2024). Tailwind CSS - A utility-first CSS framework.

<https://tailwindcss.com/docs>

Vite Documentation. (2024). Vite - Next Generation Frontend Tooling.

<https://vitejs.dev/>

Python-Jose Documentation. (2024). Python-Jose - JavaScript Object Signing and Encryption.

<https://python-jose.readthedocs.io/>

Passlib Documentation. (2024). Passlib - Password hashing library.

<https://passlib.readthedocs.io/>